

3. 배열처리(for...)

- JavaScript에서는 배열(Array)도 객체(Object)지만 객체 순회(Iterate)를 할 때 for in을 사용해서 좋을 게 없다.
- for in은 프로토타입 체인에 있는 프로퍼티를 모두 훑는(enumerate) 데다가 객체 자신의 프로퍼티만 훑으려면 hasOwnProperty를 사용해야 하기 때문에 for보다 20배 느리다
- 배열을 만들 때 배열 생성자에 파라미터를 넣어 만드는 방법은 헛갈릴수있다. 그래서 항상 각 괄호([]) 노테이션을 이용해 배열을 만들 것을 권한다
- push, pop 보다 unshift, shift가 느리다

```
var list = [1, 2, 3, 4, 5];
for(var i = 0, l = list.length; i < l; i++) {
    console.log(list[i]); // 배열의 index 찾아옴
}

var arr = [1, 2, 3, 4, 5, 6];
arr.length = 3;
console.log(arr); // [1, 2, 3]

arr.length = 6;
arr.push(4);
console.log(arr); // [ 1, 2, 3, <3 empty items>, 4 ]
```

1. For

- for 문은 객체를 순차적으로 순회 하면서 각 항목에 대한 처리를 하고 자 할 때 사용 하는 것이다.
- for 문의 유형은 for, forEach, for of, for in이 있다.
- for : 순차적으로 순회
- forEach : 반복문이 아니라 '함수'로 인자로 함수를 받아 각 배열의 요소에 해당 함수를 적용한다.
- for of : 이터러블한 객체의 순회를 도와주는 반복문
 - 배열의 각 요소를 순회
 - 문자열을 이루는 문자 요소를 순회
- for in : Object에 있는 key에 차례로 접근하는 데 사용되는 반복문

```
var lists = [1, 2, 3, 4, 5];

// C style
for(let i = 0; i < lists.length; i++) {
    console.log(lists[i]);
}

/**
 * 1
 * 2
 * 3
 * 4
```

```
* 5
**/
```

1-1. 동일한 결과를 얻기 위한 for 사용

```
// for of / in
for(let list of lists) {
  console.log(list);
}

for(let list in lists) {
  console.log(list);
}

// forEach
lists.forEach(function(item) {
  console.log(item);
})

// forEach 화살표 함수
lists.forEach((item) => console.log(item));
```

1-2. forEach를 사용한 방법

```
// item : 값
// index : position
// array : 원
lists.forEach(function(item, index, array) {
  console.log(item, index, array);
})

/**
 * 1 0 (5) [1, 2, 3, 4, 5]
 * 2 1 (5) [1, 2, 3, 4, 5]
 * 3 2 (5) [1, 2, 3, 4, 5]
 * 4 3 (5) [1, 2, 3, 4, 5]
 * 5 4 (5) [1, 2, 3, 4, 5]
 */

lists.forEach((item, index) => console.log(item, index ));
/**
 * 1 0
 * 2 1
 * 3 2
 * 4 3
 * 5 4
 */
```

2. 배열 기본

2-1. push, pop, unshift, shift

```
let arrs = [1, 2];

// 뒤에서 작업
arrs.push(3,4);
console.log(arrs); // [1, 2, 3, 4]
arrs.pop();
console.log(arrs); // [1, 2, 3]

// 앞에서 작업
arrs.unshift("앞1","앞2");
console.log(arrs); // ['앞1', '앞2', 1, 2, 3]
arrs.shift();
console.log(arrs); // ['앞2', 1, 2, 3]
```

2-2. splice, concat, indexOf, lastIndexOf, includes

```
const lists = [];
lists.push(1, 2, 3); // (3) [1, 2, 3]
console.log(lists);

// 지정한 index를 제외한 모두 삭제
lists.splice(1);
console.log(lists); // [1]

lists.push(2,3);
console.log(lists); (3) [1, 2, 3]

// 지정한 index를 삭제 하고 해당 index에 삽입
lists.splice(1, 1, 4, 5, 6);
console.log(lists); // (5) [1, 4, 5, 6, 3]

// 합치기 const addLists = [10, 11, 10];
const newList = lists.concat(addLists);
console.log(newList); // (8) [1, 4, 5, 6, 3, 10, 11, 10]

// index 찾기
console.log(newList.indexOf(10)); // 5 :: 중복이 있으면 앞에 있는 것의 index
console.log(newList.lastIndexOf(10)); // 7 :: 중복이 있으면 뒤에 있는 것의 index
console.log(newList.indexOf(2)); // -1

// 포함
console.log(newList.includes(10)); // true
console.log(newList.includes(2)); // false
```

2-3. join, split

```
const cars = ['소나타', 'SM5', '그랜저'];

// 문자열 변환
let convertString = cars.join();
console.log(convertString); // 소나타,SM5,그랜저

// 구분자 추가 하여 문자열 변환
convertString = cars.join("#");
console.log(convertString); // 소나타#SM5#그랜저

// string을 array로 변환
const carString = '소나타,SM5,그랜저';

// , 로 분리 하여 array로 변경
let convertArray = carString.split(",");
console.log(convertArray); // (3) ['소나타', 'SM5', '그랜저']
convertArray = carString.split(",", 2);
console.log(convertArray); // (2) ['소나타', 'SM5']
```

2-4. reverse

```
const cars = ['소나타', 'SM5', '그랜저'];
let carsReverse = cars.reverse();

console.log(carsReverse); // (3) ['그랜저', 'SM5', '소나타']
console.log(cars); // (3) ['그랜저', 'SM5', '소나타']
```

2-5. splice와 slice 의 차이점

```
let cars = ['소나타', 'SM5', '그랜저'];
let newCars = cars.splice(0,1); // 원본이 외곡됨

console.log(newCars); // ['소나타'] -> 삭제 된 값
console.log(cars); // (2) ['SM5', '그랜저'] -> 남아 있는 값

cars = ['소나타', 'SM5', '그랜저'];
newCars = cars.slice(1,3); // 원본이 외곡 되지 않음
console.log(newCars); // (2) ['SM5', '그랜저']

newCars = cars.slice(0,1);
console.log(newCars); // ['소나타']

newCars = cars.slice(1,2);
console.log(newCars); // ['SM5']

console.log(cars); // (3) ['소나타', 'SM5', '그랜저']
```

3. 배열 처리

- .map() : 형태를 바꿀 수 있지만 길이는 유지
- .sort() : 형태나 길이는 변경 되지 않고 순서만 바뀜
- .filter() : 길이를 변경 하지만 형태는 바뀌지 않음
- .find() : 배열을 반환 하지 않음, 한 개의 데이터가 반환되고 형태는 바뀌지 않음
- .forEach() : 형태 이용, 변환 없음

```
const arrayObj = ['1.0', '기능', '1.25'];

var convertNumber = function(obj) {
  const arrayObjTem = [];
  for(let i = 0 ; i < obj.length ; i++ ) {
    const itemValue = parseFloat(obj[i]);
    if(itemValue) { // 문자는 NaN으로 해석이 되어서 false 임
      arrayObjTem.push(itemValue);
    }
  }
  return arrayObjTem;
}

console.log(convertNumber(arrayObj)); // [1, 1.25]

arrayObj.map(item => parseFloat(item)) // [1, NaN, 1.25]
arrayObj.map(item => parseFloat(item)).filter(item=>item) // [1, 1.25]
```

4. 배열을 고급 처리

객체에서 특정 정보를 추출하는 방법에는 여러가지가 존재 하지만 여기서는 map, reduce를 통해서 알아 본다.

- map은 객체의 항목을 순환 하면서 처리를 하고
 - map() 메서드는 각 배열 요소에 대해 함수를 수행하여 새 배열을 만든다.
 - map() 메서드는 값이 없는 배열 요소에 대해 함수를 실행하지 않는다.
 - map() 메서드는 원래 배열을 변경하지 않는다.
- reduce는 객체 항목을 순환 하면서 처리를 하지만 결과는 새로 생성된 객체를 통해서 만들어 지며(추가) 된다
 - reduce() 메서드는 각 배열 요소에서 함수를 실행하여 단일 값을 생성 (축소)한다.
 - reduce() 메서드는 배열의 왼쪽에서 오른쪽으로 작동한다. reduceRight()도 참조해라.
 - reduce() 메서드는 원래 배열을 줄이지 않는다.

4-1. map 변환 과정

```
const addressArr = [
  { name: '김길자', address: '서울' },
  { name: '홍길동', address: '강원도' },
```

```

    { name: '김영이', address: '충청도' },
    { name: '바둑이', address: '서울' },
    { name: '뱀', address: '경상도' }
  ];

  const getAddress = function(arr) {
    const arrAddress = [];
    for(let i = 0 ; i < arr.length ; i++ ) {
      const address = arr[i].address;
      arrAddress.push(address);
    }
    return arrAddress;
  }

  console.log(getAddress(addressArr));
  // [ '서울', '강원도', '충청도', '서울', '경상도' ]

```

- 1차 함수로 변경

```

const getAddress = function(arr) {
  const arrAddress = [];
  for(let i = 0 ; i < arr.length ; i++ ) {
    arrAddress.push(getAddressItem(arr[i]));
  }
  return arrAddress;
}

// 함수화
var getAddressItem = function(arrItem) {
  return arrItem.address;
}

console.log(getAddress(addressArr));
// [ '서울', '강원도', '충청도', '서울', '경상도' ]

```

- map로 변경

```

console.log(addressArr.map(item => getAddressItem(item)));
// [ '서울', '강원도', '충청도', '서울', '경상도' ]

// addressArr 객체안의 객체는 하나를 의미 하므로 파라미터 생략 가능
console.log(addressArr.map(getAddressItem));
// [ '서울', '강원도', '충청도', '서울', '경상도' ]

```

4-2. reduce 처리

길이와 형태 변경 됨

- total : 합계 (초기 값 / 이전에 반환된 값)
- value : 항목 값
- index : 항목 인덱스
- array : 배열 자체

```

/*
accumulator  currentValue  currentIndex  sum    array
    0            0            0      0    [0, 1, 2, 3, 4]
    0            1            1      1    [0, 1, 2, 3, 4]
    1            2            2      3    [0, 1, 2, 3, 4]
    3            3            3      6    [0, 1, 2, 3, 4]
    6            4            4     10    [0, 1, 2, 3, 4]
*/
var sum = [0, 1, 2, 3, 4].reduce(function(accumulator,
                                         currentValue,
                                         currentIndex,
                                         array) {
    return accumulator + currentValue;
});
console.log(sum); // 10

/*
accumulator  currentValue  currentIndex  sum    array
    10          0            0      10    [0, 1, 2, 3, 4]
    10          1            1      11    [0, 1, 2, 3, 4]
    11          2            2      13    [0, 1, 2, 3, 4]
    13          3            3      16    [0, 1, 2, 3, 4]
    16          4            4      20    [0, 1, 2, 3, 4]
*/
// 두번째 파라미터는 초기 값을 의미함
var sumInit = [0, 1, 2, 3, 4].reduce(function(accumulator,
                                              currentValue,
                                              currentIndex,
                                              array) {
    return accumulator + currentValue;
}, 10);
console.log(sumInit); // 20

```

4-2-1. reduce 함수 호출

```

const fCallback = function(collectionValues, item) {
    return [ ...collectionValues, item];
};

const saying = ['abc', 'xyz', 'opq'];

```

```
const initialValue = ['zzz'];
const copy = saying.reduce(fCallback, initialValue);
console.log(copy);    // (4) ['zzz', 'abc', 'xyz', 'opq']
console.log(saying);  // (3) ['abc', 'xyz', 'opq']
```

4-3. map, filter 예제

4-3-1 배열 값에 2 곱하기 map 예제

```
var numbers1 = [1, 2, 3, 4, 5];
var numbers2 = numbers1.map(myFunction);

/**
 * value : 항목 값
 * index : 항목 인덱스
 * array : 배열 자체
 */
function myFunction(value, index, array) {
    return value * 2;
}

console.log(numbers1); // > [1, 2, 3, 4, 5]
console.log(numbers2); // > [2, 4, 6, 8, 10]
```

4-3-2. 3보다 큰 filter예제

```
var numbers = [1, 2, 3, 4, 5];
var over3 = numbers.filter(myFunction);

/**
 * value : 항목 값
 * index : 항목 인덱스
 * array : 배열 자체
 */
function myFunction(value, index, array) {
    return value > 3;
}

console.log(numbers); // > [1, 2, 3, 4, 5]
console.log(over3);  // > [4, 5]
```

4-3-3. 특정 데이터 추출

```
const cars = [
    {name: '그랜저', company: '현대'},
```



```

    {name: 'SM', company: '삼성'},
    {name: '포니', company: '현대'},
    {name: '코란도', company: '쌍용'}
  ];

  // 현대 소속 자동차 이름
  var fSearchMapCars = function(searchCompany) {
    // searchCompany 와 같지 않으면 undefined 이므로 필터를
    // 사용 해서 객체만 얻음
    return cars.map((car) => {
      if (car.company == searchCompany) {
        return car;
      }
    })
    .filter(car => car);
  };

  console.log(fSearchMapCars('현대'));
  // [ { name: '그랜저', company: '현대' }, { name: '포니', company: '현대' } ]

```

4-4. reduce 예제

```

const cars = [
  {name: '그랜저', company: '현대'},
  {name: 'SM', company: '삼성'},
  {name: '포니', company: '현대'},
  {name: 'SM6', company: '삼성'}
];

// 삼성 소속 자동차 이름
var fSearchReduceCars = function(searchKey) {
  // targetCars의 초기값은 [] 이고 조건에 만족 하는 경우 추가 되면
  // 할목별 순환 할 때 파라미터로 전달 된다. ( 합쳐진 값 )
  // car는 cars의 현재 항목 임
  return cars.reduce((targetCars, car) => {
    if (car.company != searchKey) {
      return targetCars;
    }
    return [...targetCars, car];
  }, []);
};

console.log(fSearchReduceCars('삼성'));
/**
 * [
 *   {name: 'SM', company: '삼성'},
 *   {name: 'SM6', company: '삼성'}
 * ]
 */

// 아래와 같이 변경 하여도 동일 하다.
var fSearchReduceCars = function(searchKey) {

```

```

    return cars.reduce((targetCars, car) => {
        return car.company !== searchKey ? targetCars : [...targetCars,
car];
    }, []);
}

var fSearchReduceCarsCnt = function() {
    return cars.reduce((targetCars, car) => {
        const count = targetCars[car.company] || 0;
        const val = {
            ...targetCars,
            [car.company] : count+1
        }
        console.log("===== val :: " , val );
        return val;
    }, {});
}
console.log(fSearchReduceCarsCnt()); // { '현대': 2, '삼성': 2 }

// 중간 로그
===== val :: { '현대': 1 }
===== val :: { '현대': 1, '삼성': 1 }
===== val :: { '현대': 2, '삼성': 1 }
===== val :: { '현대': 2, '삼성': 2 }

```

5. 배열 처리 예제

5.1. 다양한 For, includes, push

```

const addressArr = [
    { name: '김길자', address: '서울' },
    { name: '홍길동', address: '강원도' },
    { name: '김영이', address: '충청도' },
    { name: '바독이', address: '서울' },
    { name: '뱀', address: '경상도' }
];

var getAddressFor = function(arr) {
    const arrAddress = [];
    for(let i = 0 ; i < arr.length ; i++ ) {
        if (arr[i].address.includes('서울')) {
            arrAddress.push(arr[i]);
        }
    }
    return arrAddress;
};

var getAddressForEach = function(arr) {

```

```

    const arrAddress = [];
    arr.forEach(item => {
        if (item.address.includes('서울')) {
            arrAddress.push(item);
        }
    });
    return arrAddress;
};

var getAddressForOf = function(arr) {
    const arrAddress = [];
    for (const item of arr) {
        if (item.address.includes('서울')) {
            arrAddress.push(item);
        }
    }
    return arrAddress;
};

var getAddressForIn = function(arr) {
    const arrAddress = [];
    for (const item in arr) {
        if (arr[item].address.includes('서울')) {
            arrAddress.push(arr[item]);
        }
    }
    return arrAddress;
};

console.log(getAddressFor(addressArr));
// [ { name: '김길자', address: '서울' }, { name: '바둑이', address: '서울' } ]
console.log(getAddressForEach(addressArr));
// [ { name: '김길자', address: '서울' }, { name: '바둑이', address: '서울' } ]
console.log(getAddressForOf(addressArr));
// [ { name: '김길자', address: '서울' }, { name: '바둑이', address: '서울' } ]
console.log(getAddressForIn(addressArr));
// [ { name: '김길자', address: '서울' }, { name: '바둑이', address: '서울' } ]

```

5.2. find, filter, some, reduce, 복합

```

const cars = [
    { company: '현대자동차', carname: '소나타', reservation: true, price: 10000 },
    { company: '현대자동차', carname: '그랜저', reservation: true, price: 20000 },
    { company: '삼성자동차', carname: 'SM5', reservation: false, price: 10000 },

```

```

    { company: '기아자동차', carname: '쏘렌
토', reservation: true, price: 15000 },
];
// find :: 첫번째 찾은 요소를 리턴
let result = cars.find( (car) => car.price === 10000 );
console.log(result);
// => {company: '현대자동차', carname: '소나
타', reservation: true, price: 10000}

// filter :: 조건에 맞는 값 리턴
result = cars.filter( (car) => car.reservation );
console.log(result);
// => 0: {company: '현대자동차', carname: '소나
타', reservation: true, price: 10000}
//      1: {company: '현대자동차', carname: '그랜
저', reservation: true, price: 20000}
//      2: {company: '기아자동차', carname: '쏘렌
토', reservation: true, price: 15000}

// map :: 특정 조건이 맞는 것을 찾아서 배열로 리턴
let companys = cars.map( (car) => car.company);
console.log(companys); // ['현대자동차', '현대자동차', '삼성자동차', '기아자동차']
result = companys.filter((element, index)=>{
    return companys.indexOf(element) === index;
});
console.log(result);

```

```

// some :: 배열의 요소 하나라도 만족 하면 true
result = cars.some((car) => car.price <= 10000); // true
console.log(result);

// every :: 배열의 모든 요소가 만족 할 때
result = cars.every((car) => car.price <= 10000); // false
console.log(result);
console.clear();

// reduce :: 배열 요소 값을 누적
result = cars.reduce((prev, curr) => {
    console.log(prev, ',', curr);
    console.log('-----')
    return prev + curr.price;
}, 0); // 55000
result = cars.reduce((prev, curr) => prev + curr.price, 0);
console.log(result); // 55000

// 복합
result = cars
    .map((car)=>car.price)
    .filter( (item) => item <= 10000 )
    .reduce( (prev, curr) => prev + curr, 0);
console.log(result); // 20000

```

```
result = cars
  .map((car)=>car.price)
  .sort((a,b) => a-b) // b-a 큰것 순
  .join();
console.log(result); // 10000,10000,15000,20000
```

5-2. 배열 api

- Array.reduceRight()
- Array.every()
- Array.some()
- Array.lastIndexOf()
- Array.find()
- Array.findIndex()